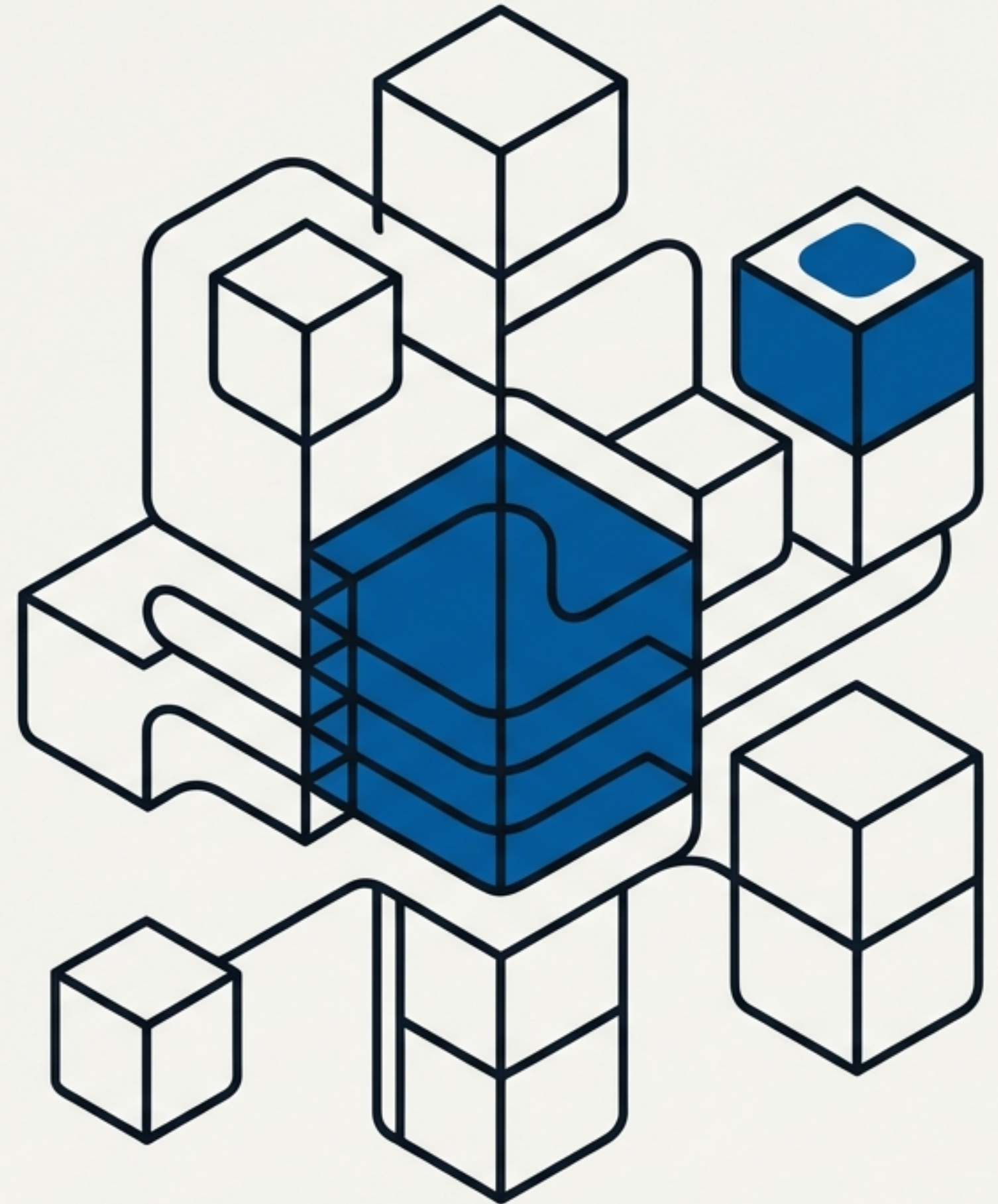
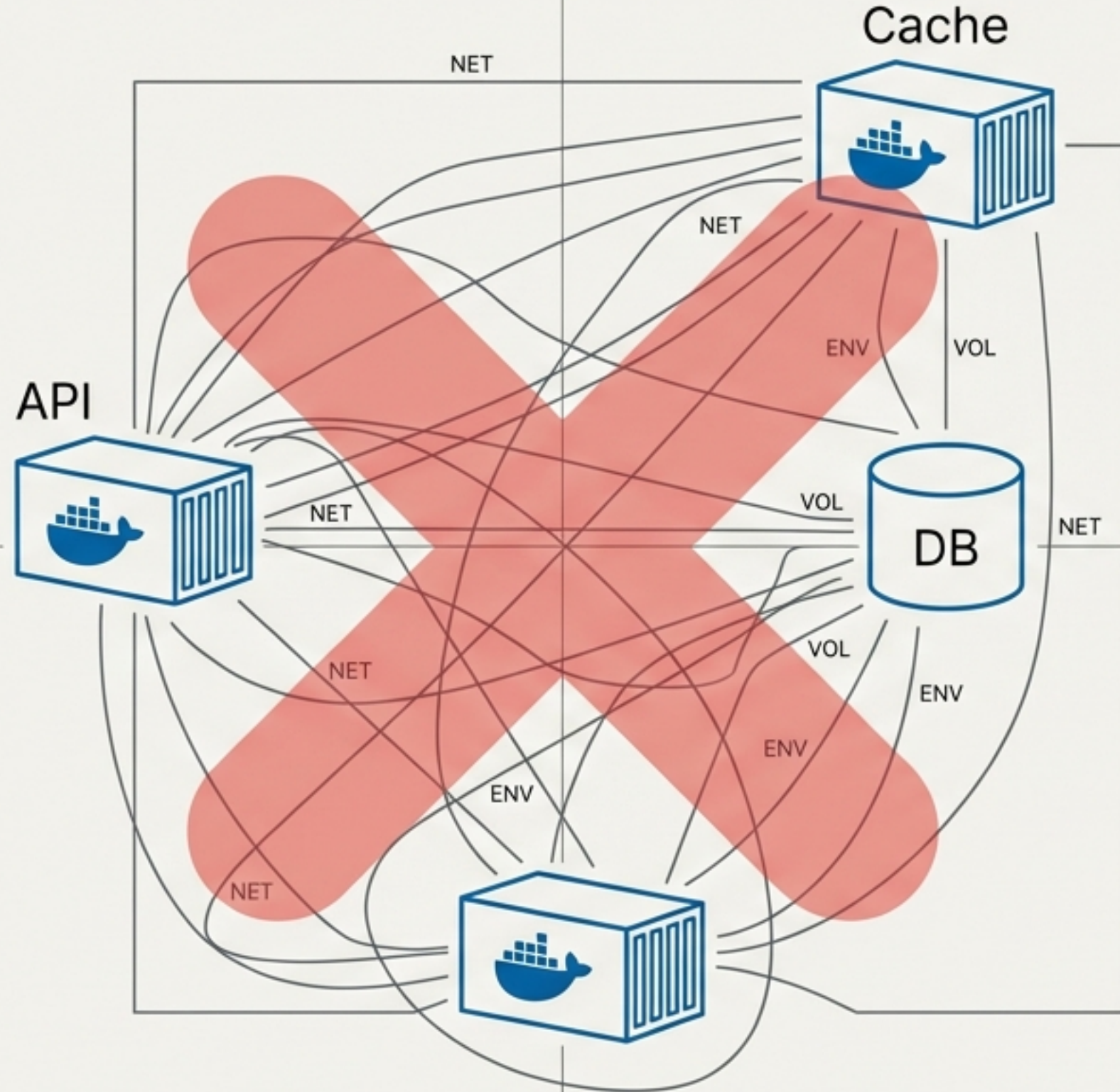


Docker Compose: A Orquestra no seu Ambiente Local

Aula 8: Transformando a complexidade de múltiplos contêineres em uma solução elegante e eficiente.

Nesta aula, você aprenderá a consolidar toda a configuração de build, volumes e redes em um Único arquivo declarativo. Nosso objetivo é claro: substituir dezenas de comandos manuais por uma abordagem que torna seus ambientes reproduzíveis e versionáveis.





O Caos Antes do Compose

Gerenciar aplicações multi-contêiner manualmente era um processo repetitivo, complexo e propenso a erros.

- ❌ Criar redes customizadas manualmente (docker network create ...)
- ❌ Definir volumes um por um (docker volume create ...)
- ❌ Executar múltiplos e longos comandos `docker run`
- ❌ Lembrar da ordem correta de inicialização dos serviços
- ❌ Configurar variáveis de ambiente repetidamente para cada contêiner

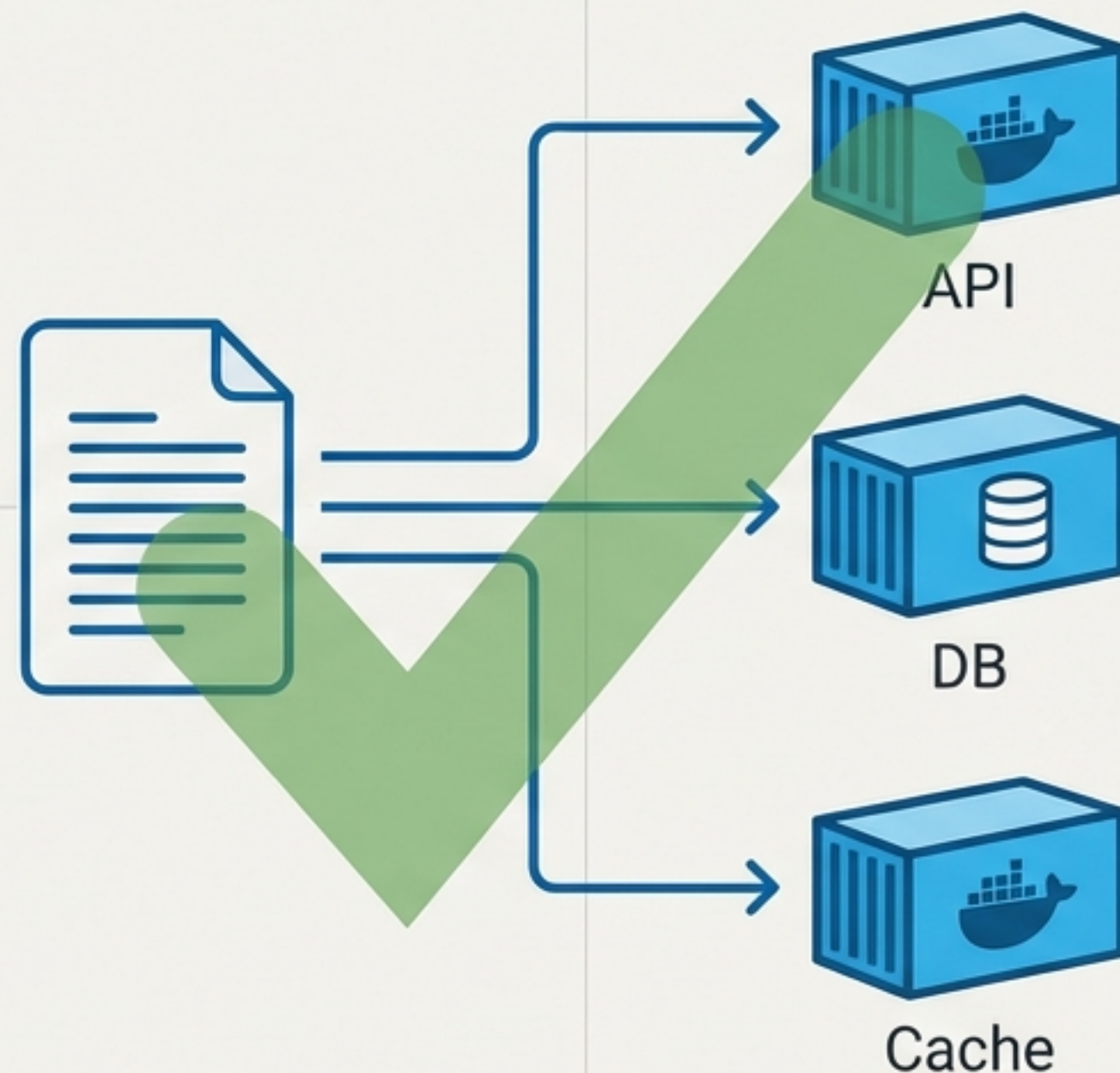
O resultado eram scripts complexos de manter e uma documentação extensa e frágil.

O Controle com o Compose: Infraestrutura como Código

O Compose introduz uma abordagem declarativa que centraliza toda a configuração do seu stack de aplicação.

- ✓ **Um único arquivo:** `docker-compose.yml` define todo o ambiente.
- ✓ **Um único comando:** `docker-compose up` orquestra tudo.
- ✓ **Gerenciamento automático:** Dependências, redes e volumes são criados e conectados automaticamente.
- ✓ **Versionamento:** A configuração da sua infraestrutura vive no Git, junto com seu código.

O resultado é um ambiente de desenvolvimento totalmente reproduzível, versionável e automatizado.



A Planta Baixa da sua Aplicação: Anatomia do `docker-compose.yml`



O arquivo `docker-compose.yml` usa a sintaxe YAML para definir a arquitetura da sua aplicação de forma legível e hierárquica.

Estrutura Fundamental

- 1 **`version`:** (Topo do Arquivo) Especifica a versão do formato do Compose. Ex: '3.8'
- 2 **`services`:** (O Coração) Seção principal onde cada contêiner é definido como um 'serviço'.
- 3 **`volumes`:** (Persistência) Define volumes nomeados que podem ser compartilhados entre serviços.
- 4 **`networks`:** (Conectividade) Declara redes customizadas para isolar e conectar serviços.

```
version: '3.8'

services:
  web:
    # configurações do serviço web...
  db:
    # configurações do serviço db...

volumes:
  dados_persistentes:
    # configurações do volume...

networks:
  minha_rede:
    # configurações da rede...
```


Definindo um Serviço: Os Blocos de Construção

Cada serviço no Compose representa um contêiner configurado, consolidando todas as opções da linha de comando de forma estruturada.

image` vs. `build`

- **image:** Puxa uma imagem pronta de um registry. (Ex: `image: postgres:14`)
- **build:** Constrói uma imagem localmente a partir de um Dockerfile. (Ex: `build: ./app`)

ports

- Expõe portas do contêiner para o host.
- Sintaxe: `"porta_host:porta_container"` (Ex: `"8080:80"`)

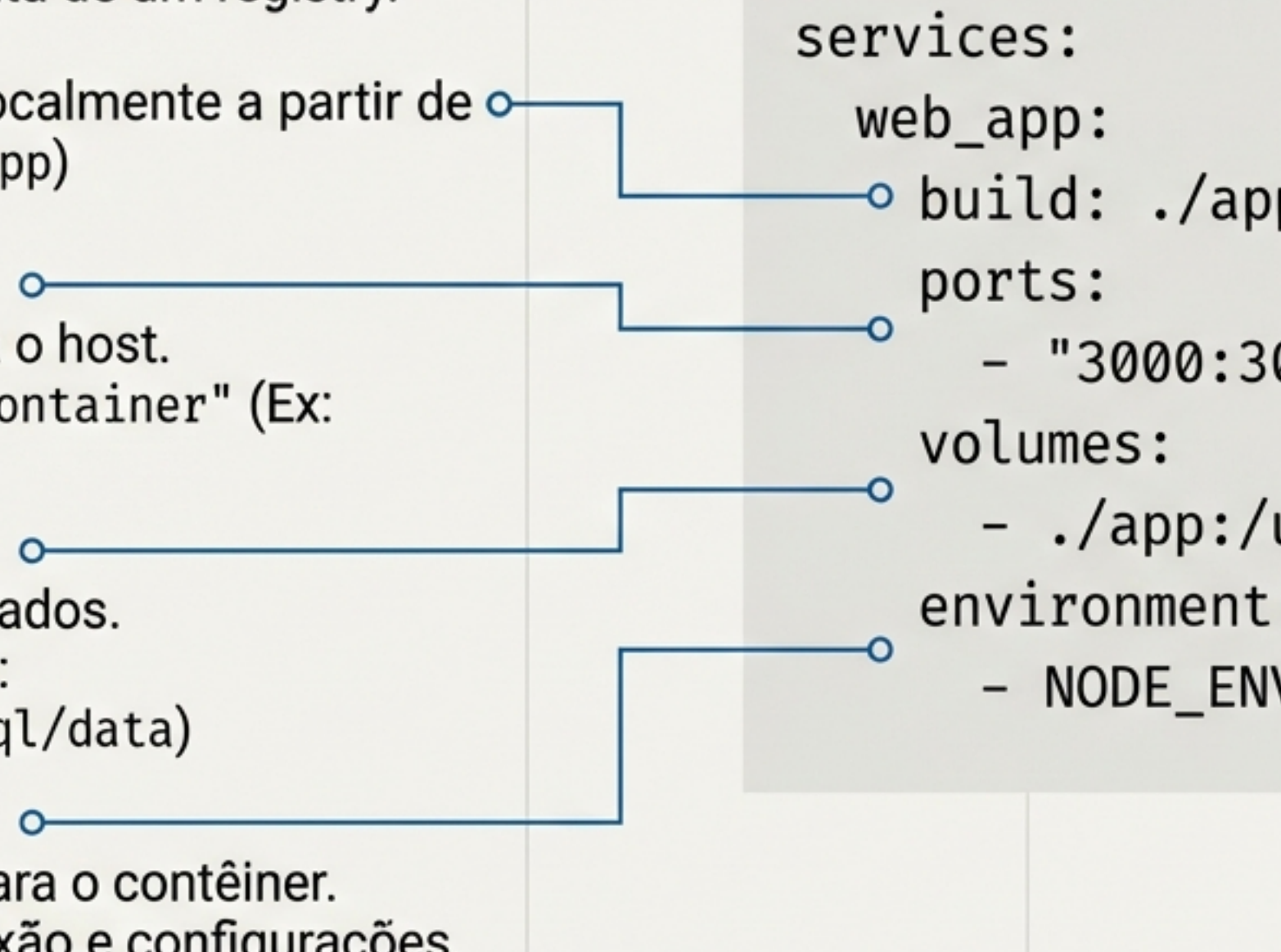
volumes

- Monta volumes para persistir dados.
- Sintaxe: `"origem:destino"` (Ex: `dados_db:/var/lib/postgresql/data`)

environment

- Define variáveis de ambiente para o contêiner.
- Útil para senhas, URLs de conexão e configurações.

```
services:
  web_app:
    build: ./app
    ports:
      - "3000:3000"
    volumes:
      - ./app:/usr/src/app
    environment:
      - NODE_ENV=development
```



Comparativo de Comandos: A Diferença é Notável

Docker CLI (Manual e Imperativo) ❌

```
# 1. Criar rede
docker network create minha_rede

# 2. Criar volume
docker volume create dados_db

# 3. Iniciar banco de dados
docker run -d --name db_postgres \
  --network minha_rede \
  -v dados_db:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=senha \
  postgres:14

# 4. Iniciar aplicação
docker run -d --name web_app \
  --network minha_rede \
  -p 3000:3000 \
  -e DATABASE_URL=postgres://... \
  minha_imagem:latest
```

Docker Compose (Declarativo e Simples) ✅

```
# docker-compose.yml
version: '3.8'
services:
  db_postgres:
    image: postgres:14
    environment:
      - POSTGRES_PASSWORD=senha
    volumes:
      - dados_db:/var/lib/postgresql/data
  web_app:
    image: minha_imagem:latest
    ports:
      - "3000:3000"
    depends_on:
      - db_postgres
    environment:
      - DATABASE_URL=postgres://...
volumes:
  dados_db:

# Comando único
$ docker-compose up -d
```

A abordagem do Compose não apenas reduz a quantidade de comandos, mas torna a infraestrutura reproduzível, versionável e documentada automaticamente.

Integração com o Desenvolvimento: A Diretiva `build`

Construa imagens personalizadas diretamente do seu código-fonte, integrando perfeitamente o processo de desenvolvimento.

Sintaxe Básica

A pasta especificada deve conter um `Dockerfile`.

```
services:
  web_app:
    build: ./app
```

Sintaxe Avançada

Permite especificar um `Dockerfile` customizado, contexto e argumentos de build.

```
services:
  web_app:
    build:
      context: ./app
      dockerfile: Dockerfile.dev
      args:
        - BUILD_ENV=development
```

Comandos e Dicas

****Forçar Reconstrução****

Use `docker-compose up --build` para forçar a reconstrução das imagens após alterações no `Dockerfile` ou código-fonte.

****Cache Inteligente****

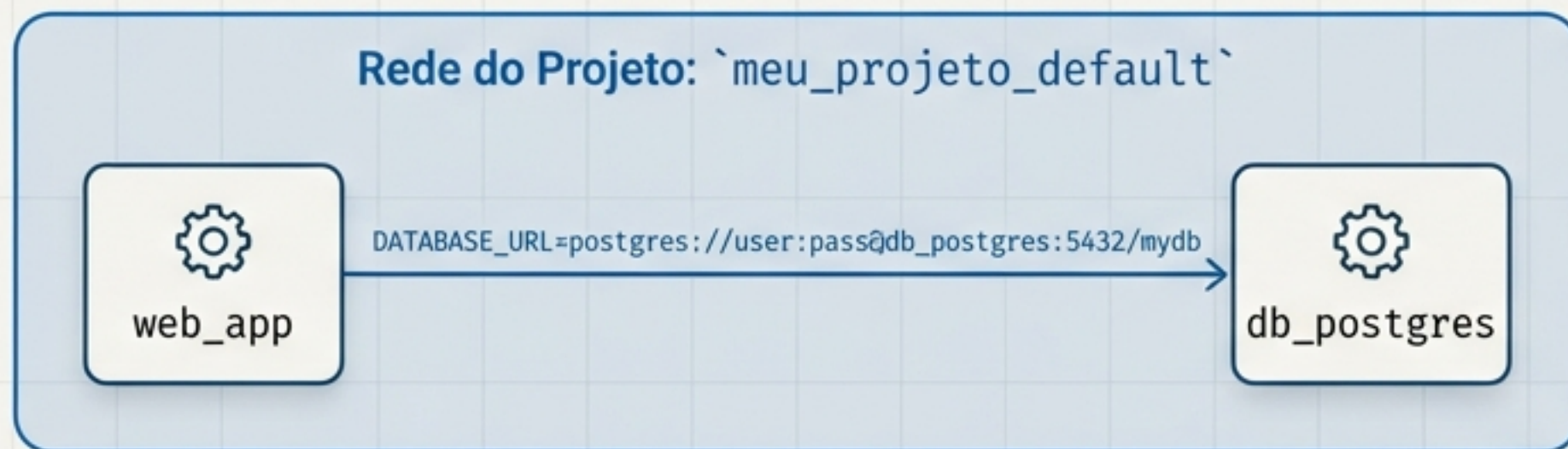
O Compose reutiliza imagens já construídas por padrão, acelerando startups subsequentes.

Conectividade Mágica: Redes e DNS Interno

O Compose gerencia redes automaticamente, criando um ambiente isolado onde seus serviços se comunicam de forma simples e segura.

Funcionalidades Chave

- ✓ **Rede Padrão Automática:** O Compose cria uma rede customizada para o projeto (ex: `meu\_projeto\_default`), isolando-o de outros.
- ✓ **DNS Interno Integrado:** Cada serviço pode ser alcançado usando seu nome como hostname. Chega de procurar por IPs.



Exemplo Prático de Comunicação

```
services:
  web_app:
    build: ./app
    environment:
      - DATABASE_URL=postgres://user:pass@db_postgres:5432/mydb
    depends_on:
      - db_postgres
  db_postgres:
    image: postgres:14
    environment:
      - POSTGRES_PASSWORD=senha_segura
```

Diagrama de conexão: Uma seta azul parte da string `postgres://user:pass@db_postgres:5432/mydb` no código do `web_app` e aponta para o serviço `db_postgres` na lista de serviços.



Pro-Tip: "Dica Profissional": Você pode definir redes customizadas na seção `networks` para criar topologias mais complexas e controlar o isolamento entre grupos de serviços.

Persistência de Dados Simplificada: Gerenciando Volumes

A estrutura em duas partes do Compose garante que seus dados sobrevivam além do ciclo de vida dos contêineres.

Nível 1: Declaração no Serviço

Especifica onde o volume será montado dentro do contêiner.



Nível 2: Definição na Raiz

Cria o volume nomeado, que é gerenciado pelo Docker.

```
services:
  db_postgres:
    image: postgres:14
    volumes:
      - dados_db:/var/lib/postgresql/data
```

```
volumes:
  dados_db:
    driver: local
```

Tipos de Montagem

- **Volume Nomeado:** ``dados_db:/caminho`` – Gerenciado pelo Docker, a melhor prática para persistência.
- **Bind Mount:** ``./local:/caminho`` – Vincula um diretório do host, ideal para desenvolvimento com hot-reload.

Comando de Limpeza (com alerta)



Para remover volumes junto com o stack:
``docker-compose down -v``

****Atenção*:** Este comando apaga permanentemente os dados nos volumes nomeados. Use com cautela.

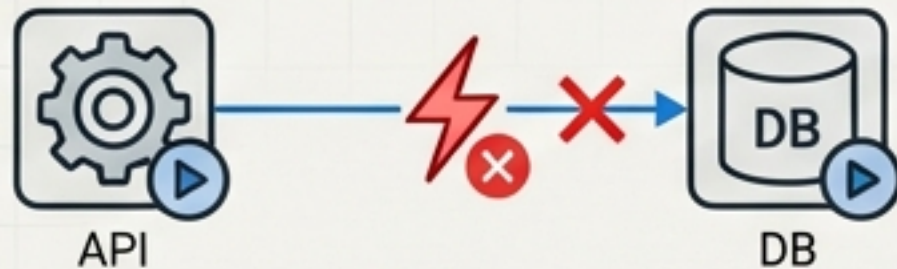
Orquestrando a Inicialização: A Diretiva `depends\_on`

Em aplicações complexas, a ordem de inicialização é crítica. `depends\_on` ajuda a controlar essa sequência.

O Problema e a Solução

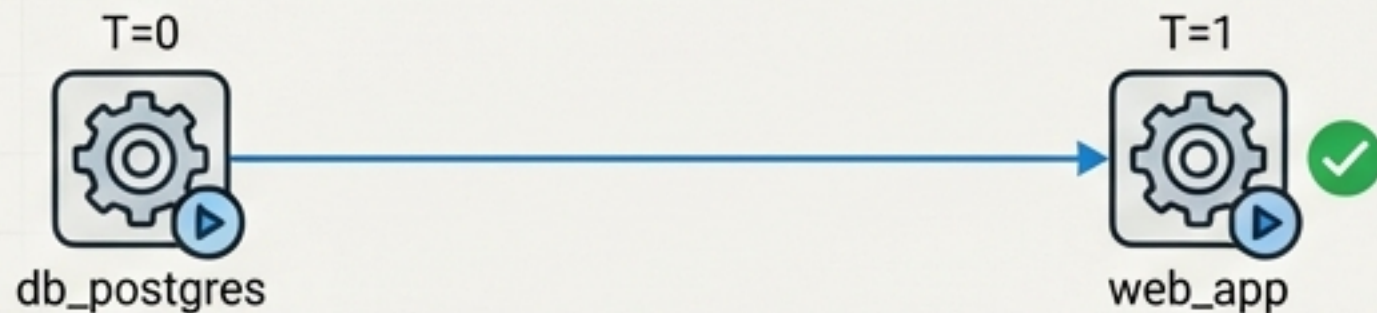
Sem `depends\_on`:

Serviços iniciam simultaneamente, podendo causar falhas de conexão se, por exemplo, a API subir antes do banco de dados.



Com `depends\_on`:

O Compose aguarda o contêiner `db\_postgres` iniciar antes de iniciar o contêiner `web\_app`.



Exemplo de Código

```
services:
  web_app:
    build: ./app
    ports:
      - "3000:3000"
    depends_on:
      - db_postgres
    environment:
      - DATABASE_URL=postgres://user:pass@db_postgres:5432/mydb

  db_postgres:
    image: postgres:14
    # ...
```

Limitação Importante

`depends\_on` apenas garante que o contêiner foi **iniciado**, não que o serviço dentro dele está **pronto** para aceitar conexões. Para aplicações robustas, implemente health checks (como no exemplo completo) ou lógicas de `retry` na sua aplicação.

Dominando o Ciclo de Vida: Comandos Essenciais



`docker-compose up`

Cria, constrói (se necessário) e inicia todos os serviços.

- d: Executa em modo 'detached' (background).
- build: Força a reconstrução das imagens.



`docker-compose ps`

Lista todos os contêineres do projeto, exibindo status, portas e nomes.

Útil para verificar rapidamente o estado do stack.



`docker-compose logs`

Exibe os logs de todos os serviços.

- f: Segue os logs em tempo real ('follow').

`docker-compose logs web_app`: Mostra logs de um serviço específico.



`docker-compose stop`

Para os contêineres em execução sem removê-los. Preserva volumes e redes.



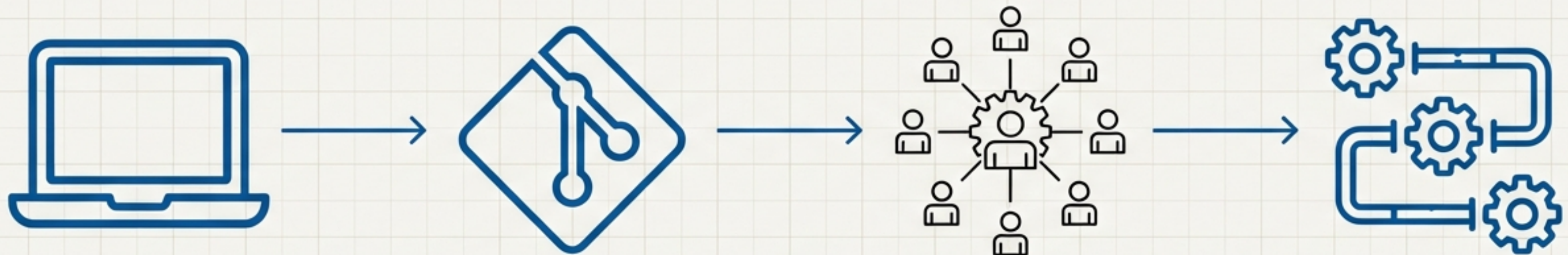
`docker-compose down`

Para e remove contêineres, redes e, opcionalmente, volumes.

- v: Remove também os volumes nomeados definidos no arquivo. Limpeza completa.

Adotando um Fluxo de Trabalho Profissional com Compose

Adotar o Compose transforma não apenas a execução de contêineres, mas todo o seu ciclo de desenvolvimento.



Desenvolvimento Local

Use bind mounts
(`./app:/usr/src/app` in Fira Code) para 'hot-reloading' de código.

Use variáveis de ambiente para diferenciar configurações de dev e produção.

Versionamento no Git

Faça o commit do seu
``docker-compose.yml`` (in Fira Code) junto com o código-fonte.

Importante: Mantenha segredos (senhas, chaves de API) em um arquivo ``.env`` e adicione o ``.env`` ao seu ``.gitignore``.

Colaboração em Equipe

Novos desenvolvedores precisam apenas de um comando:
``docker-compose up`` (in Fira Code).

Zero configuração manual.

Testes e Integração Contínua (CI)

Use o mesmo ``.docker-compose.yml`` (in Fira Code) em seus pipelines de CI/CD para garantir total paridade entre os ambientes de teste e desenvolvimento.

Boas Práticas para Arquivos Compose Robustos e Seguros



Organize por Ambiente

Mantenha arquivos separados (`docker-compose.yml` , `docker-compose.dev.yml`) e combine-os com o comando `docker-compose -f docker-compose.yml -f docker-compose.dev.yml up`.



Versione Suas Imagens

Evite usar a tag `:latest` . Especifique versões concretas (ex: `postgres:14.5`) para garantir builds reproduzíveis e evitar surpresas.



Nunca Versione Segredos

Use variáveis de ambiente carregadas de um arquivo `.env` (ignorado pelo Git) ou use Docker Secrets para produção.



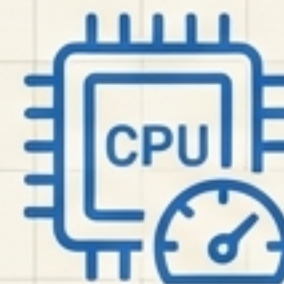
Documente Dependências e Configurações

Use `depends_on` para expressar relações e adicione comentários (`#`) no YAML para explicar configurações complexas.



Nomeie Seus Volumes

Sempre defina volumes nomeados na seção raiz. Volumes anônimos são difíceis de gerenciar e podem levar à perda de dados.



Limite o Uso de Recursos

Para ambientes compartilhados, use `deploy.resources.limits` para prevenir que um serviço consuma toda a CPU ou memória do host.

Exemplo Completo: Um Stack Web Moderno

Vamos consolidar tudo em um exemplo prático: uma aplicação com banco de dados, cache, e API, um padrão arquitetural comum.

```
version: '3.8'
```

```
services:
```

```
  db: # Serviço de Banco de Dados
```

```
    image: postgres:14.5
```

```
    environment:
```

```
      POSTGRES_DB: meu_app
```

```
      POSTGRES_USER: admin
```

```
      POSTGRES_PASSWORD: senha_segura
```

```
    volumes:
```

```
      - dados_postgres:/var/lib/postgresql/data
```

```
    networks:
```

```
      - backend
```

```
    healthcheck:
```

```
      test: ["CMD-SHELL", "pg_isready -U admin"]
```

```
      interval: 10s
```

```
      timeout: 5s
```

```
      retries: 5
```

```
  cache # Serviço de Cache em memória
```

```
    image: redis:7-alpine
```

```
    command: redis-server --appendonly yes
```

```
    volumes:
```

```
      - dados_redis:/data
```

```
    networks:
```

```
      - backend
```

Garante que o banco está pronto

Definição dos volumes nomeados

```
  api: # Serviço da API Backend
```

```
    image: redis:7-alpine
```

```
    command: redis-server --appendonly yes
```

```
    volumes:
```

```
      - dados_redis:/data
```

```
    networks:
```

```
      - backend
```

```
  api: # Serviço da API Backend
```

```
    build: ./backend
```

```
    ports: ["4000:4000"] # Exemplo de porta
```

```
    depends_on:
```

```
      db:
```

```
        condition: service_healthy
```

```
      cache:
```

```
        condition: service_started
```

```
    networks:
```

```
      - backend
```

```
  volumes:
```

```
    dados_postgres:
```

```
    dados_redis:
```

```
  networks:
```

```
    backend:
```

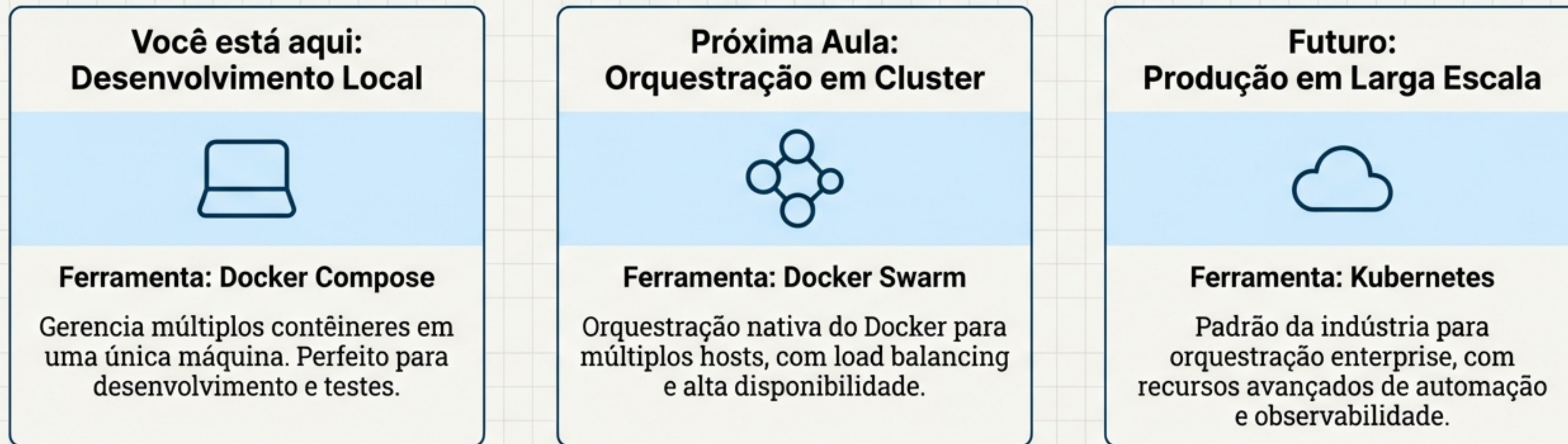
Espera o healthcheck passar

Definição da rede customizada

Sua Jornada Continua: Do Ambiente Local ao Cluster

Você dominou a orquestração local. Agora, vamos ver o que vem a seguir para ambientes de produção em escala.

Diagrama de Progressão



****🎯 Tarefa Final da Aula 8****: Complete o exercício prático `TF008` disponível no portal.

****Preparação****: Revise os conceitos de volumes e redes das aulas anteriores, pois eles são fundamentais para entender o Docker Swarm.